

CO3 AT2

Debugging Tool.

Aswini Prasad R

192524077

CSA1403

Compiler Design

19/08/2026.

1) Debug syntax directed definition:-

$$E \rightarrow E_1 + T$$

$$E \rightarrow T$$

$$T \rightarrow \text{num.}$$

step 4:- Error identification

$$E \rightarrow E_1 + T \quad \{E.\text{val} = T.\text{val}\}$$

The rule ignore value of left operand E_1 .

Root cause :-

$$E \rightarrow E_1 + T \text{ is: } E.\text{val} = E_1.\text{val} + T.\text{val}$$

However, given rule calculates only;

$$E.\text{val} = T.\text{val}$$

$$E.\text{val} = 5$$

For exp $3+5$, the value of 3 is stored in $E_1.\text{val} = 3$, and the value of 5 is stored in $T.\text{val} = 5$.

corrected :-

$$E \rightarrow E_1 + T \quad \{E.\text{val} = E_1.\text{val} + T.\text{val}\}$$

$$E \rightarrow T \quad \{E.\text{val} = T.\text{val}\}$$

$$T \rightarrow \text{num} \quad \{T.\text{val} = \text{num.lexval}\}$$

Attribute Evaluation:-

For $3+5$:

$$T \rightarrow \text{num}$$

$$T.\text{val} = 3$$

$$T \rightarrow \text{num}$$

$$T.\text{val} = 5$$

$$E \rightarrow E_1 + T$$

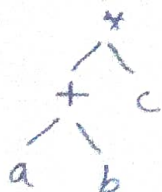
$$E_1.\text{val} = 3$$

$$T.\text{val} = 5$$

$$E.\text{val} = 3 + 5 \\ = 8$$

2) Debugging syntax tree:-

$$a + b * c$$



The compiler generates:-

$$d_1 = a + b$$

$$d_2 = d_1 * c$$

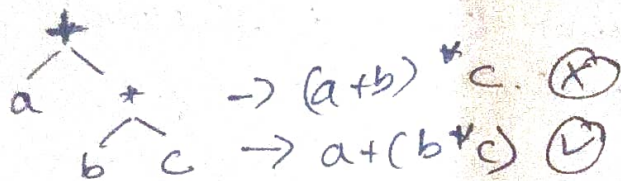
error identification:-

$$a + b * c$$

$$a + (b * c) \text{ not } (a + b) * c$$

Root cause:- $\rightarrow$ incorrect place "+"

corrected tree is



$$d_1 = a + b$$

$$d_2 = d_1 * c$$

$\} \rightarrow (a + b) * c \rightarrow \text{incorrect}$

Thus

$$d_1 = b * c$$

$$d_2 = a + d_1$$

$\rightarrow a + b * c$ is

translated as:

$$d_1 = b * c$$

$$d_2 = a + d_1$$

3) Debug Annotated Parse tree:-

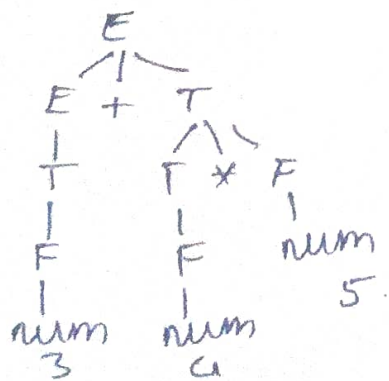
$$3 + 4 * 5$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow \text{num}$$

$$[E.\text{Val} = 35]$$



conceptually:-



$$F \rightarrow \text{num}$$

we get:-

$$F.\text{Val} = 3$$

$$F.\text{Val} = 4$$

$$F.\text{Val} = 5$$

For :-

$$T \rightarrow T * F$$

$$T.val = 4 * 5 = 20$$

Then -

$$E \rightarrow E + T$$

$$E.val = 3 + 20 = 23$$

The compiler reports :-

$$E.val = 35$$

$$\text{But: } 3 + 4 * 5 = 3 + 20 \\ = 23$$

So, 35 is incorrect.

$$(3 + 4) * 5 = 35$$

4) Debug type checking process :-

$$a + b * c$$

$a + (b * c)$ $\rightarrow$ * higher precedence.

Analyze $b * c$

Types :-

b: float

c: int.

So, $c: \text{int} \rightarrow \text{float} \Rightarrow \underline{\text{Result}}: \text{float}$

Thus:-

$b * c: \text{float}$.

Step 3:- $a + (b * c)$

$a: \text{int} \rightarrow \text{float}$

$\text{float}(a) + (b * c) \rightarrow \text{float}$

$$t_1 = \text{inttofloat}(c)$$

$$t_2 = b * t_1$$

$$t_3 = \text{inttofloat}(a)$$

$$t_4 = t_3 + t_2$$

$$x = t_4$$

⇒ Debug type checking result:

int a;
float b;
int x;

statements:

x = a + b;
b = a + x; → valid.

x = a + b.
↓ ↓
int float.

float(a) + b → conceptually.

x = a + b.
↓ ↓
int float. } float → int. ⇒ rejected because it may lose fractional part.

Ex: a = 5
b = 2.5 } → a + b = 7.5

a: int
x = int a + x: int.

b = a + x
↓ ↓
float int } → int → float

conceptually:-

b = float(a + x).

Corrected Analysis:-

statement	exp type	LHS type	req. conv.	Result
x = a + b	float	int	float → int	Invalid
b = a + x	int	float	int → float	valid.